

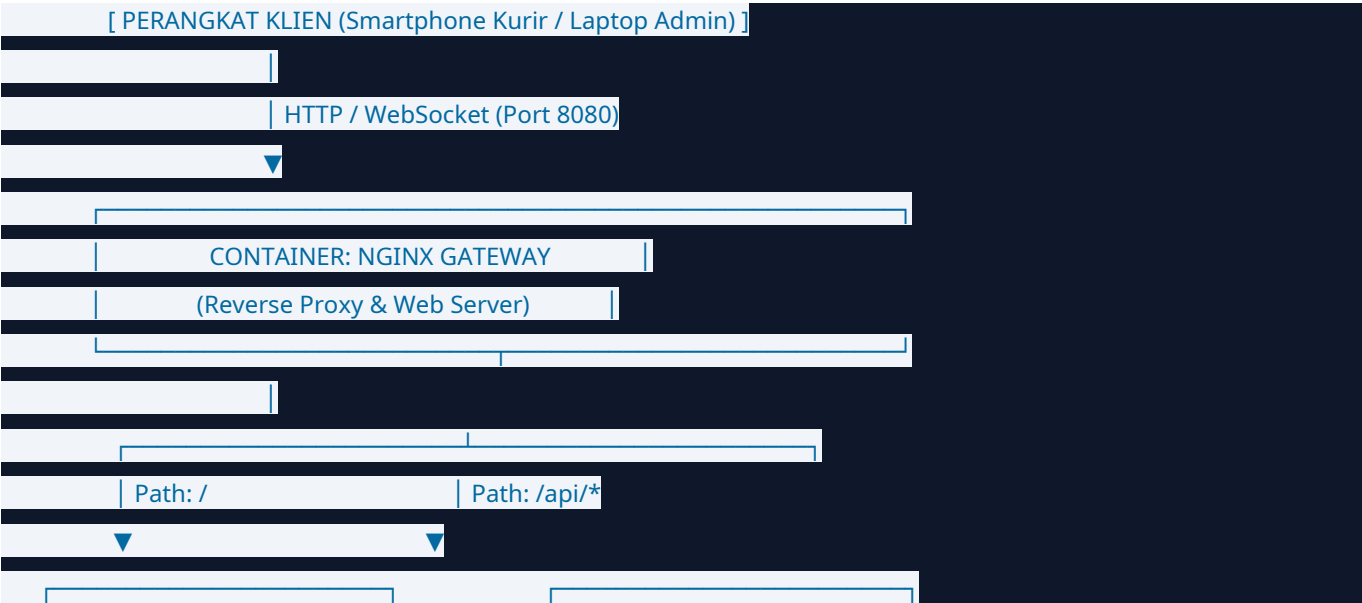
Arsitektur Stack & Sistem Operasional — Dijak Express

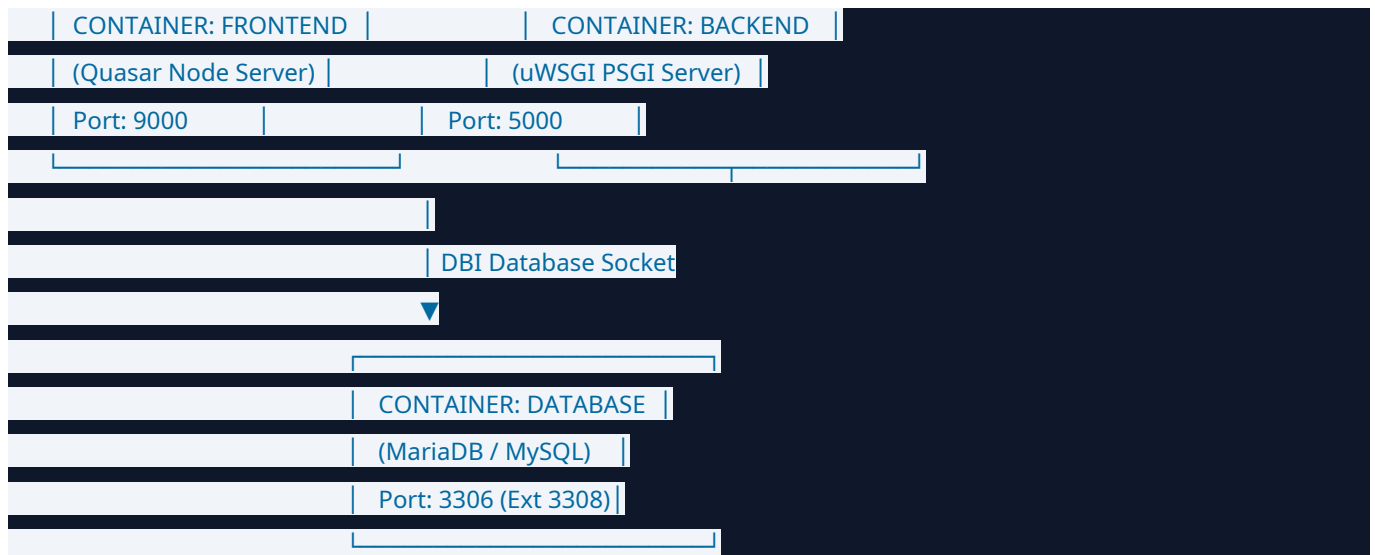
Cetak Biru Arsitektur Stack & Sistem Operasional — Dijak Express

Dokumen ini merupakan panduan spesifikasi teknis arsitektur komprehensif untuk platform logistik **Dijak Express (Scan-Resi)**. Seluruh aspek teknis diuraikan secara mendalam: mulai dari topologi kontainer Docker, tata kelola jaringan internal, mekanisme Front-End PWA, arsitektur server uWSGI, logika Back-End Perl modular, transaksi anti-duplikasi basis data, hingga kamus skema data relasional.

1. Topologi Jaringan & Arsitektur Sistem Global

Sistem mengadopsi **Arsitektur 3-Tier Enterprise Terdistribusi** yang dikontainerisasi menggunakan **Docker Compose** ke dalam 4 layanan yang beroperasi di dalam jaringan privat virtual lokal bertipe bridge ([wahana_net](#)):





Karakteristik Jaringan Virtual ([wahana_net](#)):

1. **Isolasi Mutlak (Network Isolation):** Seluruh container berjalan dalam subnet tertutup ([driver: bridge](#)). Pihak luar tidak dapat mengakses port database internal (3306) secara langsung tanpa melalui jalur aplikasi.
2. **Docker Internal DNS Resolution:** Komunikasi antar-kontainer tidak menggunakan alamat IP statis yang rentan berubah, melainkan menggunakan nama alias service:
 - Nginx memanggil backend via [http://backend:5000](#)
 - Nginx memanggil frontend via [http://frontend:9000](#)
 - Backend menghubungi database via host [db:3306](#)
3. **Port Mapping ke Host:**
 - Port [8080](#) host diarahkan ke Port [80](#) Nginx (pintu akses pengguna).
 - Port [5000](#) host dibuka untuk akses pengujian langsung ke backend uWSGI.
 - Port [3308](#) host diarahkan ke Port [3306](#) MariaDB untuk kemudahan inspeksi database developer (DBeaver/Navicat).

2. Pembedahan Mendalam Lapisan Teknologi (Technology Stack)

A. Lapisan Antarmuka Pengguna (Presentation Tier / Front-End)

Front-end dirancang responsif untuk mengakomodasi dua jenis pengguna: petugas operasional gudang dengan smartphone dan staf administrasi dengan komputer desktop:

1. **Framework Inti: Vue.js 3** menggunakan Composition API (`<script setup>`) yang dipadukan dengan **Quasar Framework v2** berbasis bundler super cepat **Vite**.
 2. **Arsitektur PWA (Progressive Web App):**
 - Dilengkapi konfigurasi Service Worker melalui Workbox (`src-pwa/`).
 - Memungkinkan aplikasi diinstal langsung pada smartphone petugas kurir tanpa melalui application store, lengkap dengan caching aset visual saat koneksi tidak stabil.
 3. **Manajemen State Terpusat (Pinia):** Dipisahkan ke dalam store modular di direktori `src/stores/`:
 - `authStore.js`: Menyimpan data sesi login, profil aktif, dan token JWT/HMAC.
 - `taskStore.js`: Mengelola data tugas shift pemindaian aktif dan riwayat kuota.
 - `scanStore.js`: Mengelola antrean pemindaian barcode real-time dan log duplikasi.
 - `paketStore.js`: Mengelola data draft paket, data pengirim/penerima, dan nomor resi.
 - `auditStore.js`: Mengelola data jejak audit untuk kebutuhan inspeksi Admin.
 4. **Dual-Mode Execution (Live API vs Mock Fallback):**
 - Sistem memiliki flag `USE_LOCAL_DATA` pada `src/services/api.js`.
 - Jika backend sedang offline atau mode demo, service layer dapat beralih secara transparan menggunakan penyimpanan memori lokal tanpa mengubah kode komponen antarmuka sedikit pun.
 5. **Mesin Pemrosesan Barcode:**
 - `html5-qrcode`: Mengakses stream kamera perangkat melalui WebRTC API, memproses frame video secara real-time pada 10–15 FPS untuk mengenali pola garis barcode 1D (Code 128) dan 2D (QR Code).
 - `jsbarcode`: Mengonversi teks nomor resi 8-karakter menjadi grafis barcode vektor berbasis SVG standar Code 128 dengan rasio kontras tinggi, siap dicetak langsung ke printer thermal label paket.
-

B. Lapisan Gateway & Reverse Proxy (Nginx)

Nginx bertindak sebagai pelindung dan pengatur lalu lintas utama ([nginx/docker-nginx.conf](#)):

1. Pemisahan Jalur Otomatis:

- Blok [location /](#): Mengalirkan seluruh permintaan antarmuka dan file statis ke server Quasar.
- Blok [location /api](#): Mengalirkan seluruh panggilan fungsi data ke Application Server uWSGI.

2. **Eliminasi Masalah CORS (Cross-Origin Resource Sharing)**: Karena Front-End dan Back-End berada di bawah satu domain dan port gerbang yang sama (Port 80/8080), browser tidak memicu pemblokiran CORS policy.
 3. **Kompresi Kinerja (Gzip Compression)**: Mengompresi respons tipe teks, JSON, CSS, dan JavaScript sebelum dikirim melalui internet untuk menghemat kuota transmisi data di area gudang dengan sinyal lemah.
 4. **Dukungan WebSocket (HMR)**: Mendukung upgrade koneksi HTTP ke WebSocket untuk fitur *Hot Module Replacement* selama proses pengembangan front-end.
-

C. Lapisan Application Server (uWSGI + PSGI Engine)

Salah satu keunggulan arsitektur proyek ini adalah penggunaan **uWSGI** dengan plugin **PSGI** (*Perl Web Server Gateway Interface*) sebagai pengganti skrip CGI biasa:

1. Arsitektur Multi-Worker & Multi-Threading ([backend/uwsgi.ini](#)):

```
master = true
workers = 4
threads = 2
enable-threads = true
```

- uWSGI menyalakan 1 proses Master dan 4 proses Worker independen. Tiap worker menjalankan 2 thread pemrosesan.
- Hal ini memungkinkan server melayani **hingga 8 transaksi pemindaian barcode secara paralel murni** dalam waktu bersamaan tanpa saling tunggu (*non-blocking*).

2. Proteksi Memory Leak & Crash Monitoring:

- `max-requests = 5000`: Worker akan dimatikan dan diregenerasi secara otomatis setelah melayani 5.000 request untuk mencegah penumpukan memori.
- `harakiri = 60`: Menjadi *watchdog timer* yang otomatis mematikan proses jika sebuah request tersangkut lebih dari 60 detik (misalnya akibat dead-lock database).

3. Hot Reloading Pengembangan:

- `touch-reload = lib/Wahana/Router.pm`: Memantau perubahan file router backend dan me-reload aplikasi secara instan tanpa perlu mematikan container.

4. Adapter PSGI (`backend/web.pl`):

- Bertindak sebagai adapter standar yang menerima struktur hash lingkungan HTTP dari uWSGI (`$env`), mengurai query parameter, membaca body JSON, mengekstrak header otentikasi, lalu menyerahkannya ke router aplikasi.

D. Lapisan Back-End Core Framework (Perl Modular)

Sistem backend ditulis dalam bahasa Perl 5 dengan pendekatan modular berkinerja tinggi tanpa dependensi framework eksternal yang berat:

1. Dispatcher Rute Berbasis Regex (`backend/lib/Wahana/Router.pm`):

- Rute didefinisikan dalam tabel `@ROUTES` berisi pasangan method HTTP, pola Regex, pointer fungsi handler, dan metadata otentikasi.
- Contoh konfigurasi rute:

```
[ 'PATCH', qr{^/api/tasks/([^\s]+)/complete$}, \&Wahana::Controller::TasksController::complete, { auth => 1 } ]
```

2. Middleware Otentikasi & RBAC (`Wahana::Auth` & `Wahana::UsersController`):

- Memeriksa keberadaan header `Authorization: Bearer <token>`.
- Mengamankan hak akses berdasarkan Role: `CUSTOMER`, `PETUGAS_SCAN`, dan `ADMIN`.

3. Pola Pemisahan Kueri Database (External Query Catalog):

- **Filosofi Desain:** Kode controller dilarang keras menuliskan query SQL secara *hard-coded*.
- Seluruh query SQL disimpan terpusat di `backend/db/query.sql` dengan penanda nama terstruktur:

```
-- name: tasks_mark_completed
UPDATE tasks SET status = 'SELESAI', completed_at = NOW() WHERE task_id = ?;
```

- Modul `Wahana::Query` mem-parsing file ini saat aplikasi boot, menyimpannya ke memori hash `%QUERIES`, dan mengeksekusinya via *Prepared Statements* dengan pengikatan parameter (`?`), menjamin keamanan mutlak dari serangan **SQL Injection**.

4. Audit Trail Otomatis (`Wahana::Audit`):

- Setiap aktivitas krusial (pembentukan resi draft, login, penyelesaian task, scan ditolak) otomatis dicatat ke tabel `audit_logs` bersama identitas pengguna, IP address, dan penanda waktu WIB.

3. Fitur Kunci & Logika Operasional Bisnis

A. Algoritma Pembuatan Resi Unik (Base32 Safe Alphanumeric)

Nomor resi dibuat secara otomatis di backend (`PaketController.pm`) dengan aturan: 1. **Penyaringan**

Karakter Ambigu: Menggunakan 32 karakter `ABCDEFGHJKLMNPQRSTUVWXYZ23456789`. Karakter ambigu `I`, `O`, `0`, dan `1` dihilangkan agar tidak membingungkan kurir saat membaca label fisik. 2. **Entropi**

Kombinasi: Panjang tetap 8 digit menghasilkan $32^8 \approx 1,09$ triliun kemungkinan kode unik. 3.

Rejection Sampling: Sistem melakukan looping hingga 20 kali percobaan dengan memeriksa keunikan nomor resi ke database (`paket_check_resi_exists`) sebelum menyimpan baris paket berstatus awal `DRAFT`.

B. Mesin Pemindaian Live & Anti-Duplikasi Transaksional

Pada saat petugas memindai barcode di gudang (`ScansController.pm`): 1. **Validasi Eksistensi:** Server memeriksa apakah nomor resi terdaftar di tabel `paket` dan berstatus `TERDAFTAR`. Resi palsu/belum lengkap langsung ditolak (`UNKNOWN_RESI`) disertai suara peringatan (*buzzer* nada rendah). 2. **Integritas**

Transaksi Database (BEGIN ... COMMIT): * Server mengunci baris data tugas shift yang bersangkutan (tasks). * Memeriksa apakah resi tersebut sudah pernah discan dalam tugas ini (scans_check_duplicate_in_task). * **Jika Duplikat:** Dicatat sebagai event DUPLICATE, penambahan kuota ditolak, dan aplikasi memunculkan peringatan audio duplikasi. * **Jika Sukses:** Dicatat sebagai event SUCCESS, progres tugas dinaikkan + 1, transaksi di-commit, dan antarmuka HP membunyikan suara BEEP nada tinggi serta kilatan visual hijau.

C. Keamanan Autentikasi & OTP 2-Faktor

- 1. **Password Hashing:** Menggunakan format Salted SHA-256:

```
sha256$<salt>$<sha256_hex(salt + password)>
```

- 2. **OTP 2FA:** Tabel user_otps menyimpan kode verifikasi sekali pakai dengan masa kedaluwarsa 5 menit dan pelacakan percobaan (attempt count) untuk mencegah serangan brute-force.

4. Kamus Data Skema Basis Data Relasional (Data Dictionary)

Sistem menggunakan database relasional MariaDB 10.11 / MySQL dengan skema yang dinormalisasi pada backend/db/schema.sql:

1. Tabel users (Manajemen Pengguna)

Kolom	Tipe Data	Keterangan
id	VARCHAR(32)	Primary Key (contoh: USR-ADMIN-001)
name	VARCHAR(100)	Nama lengkap pengguna
username	VARCHAR(50)	Nama pengguna unik (Unique Key)
email	VARCHAR(100)	Alamat email (opsional / untuk OTP)
password_hash	VARCHAR(255)	Hash password terenkripsi salted SHA-256

Kolom	Tipe Data	Keterangan
role	ENUM	Hak akses: 'ADMIN', 'PETUGAS_SCAN', 'CUSTOMER'
status	ENUM	Status akun: 'ONLINE', 'OFFLINE', 'DISABLED'
last_login	DATETIME	Waktu login terakhir
created_at	TIMESTAMP	Waktu pembuatan akun

2. Tabel `user_otps` (Verifikasi OTP)

Kolom	Tipe Data	Keterangan
id	INT AUTO_INCREMENT	Primary Key
user_id	VARCHAR(32)	ID pengguna yang meminta OTP
email	VARCHAR(100)	Alamat email tujuan
otp_hash	VARCHAR(255)	Hash dari 6 digit kode OTP
attempt_count	INT	Jumlah percobaan input (maksimal toleransi)
used	TINYINT(1)	Status terpakai: 0 (aktif), 1 (sudah dipakai)
expires_at	DATETIME	Batas waktu kedaluwarsa
created_at	TIMESTAMP	Waktu pembuatan OTP

3. Tabel **tasks** (Alokasi Shift Petugas)

Kolom	Tipe Data	Keterangan
task_id	VARCHAR(32)	Primary Key (contoh: TSK-001)
user_id	VARCHAR(32)	Foreign Key merujuk ke tabel users(id)
shift	ENUM	Shift kerja: 'Pagi', 'Sore'
tanggal	DATE	Tanggal penugasan
target	INT	Target kuota pemindaian (default: 100)
progress	INT	Jumlah paket yang berhasil discan
status	ENUM	Status tugas: 'DRAFT', 'PROSES_SCAN', 'SELESAI'
lokasi	VARCHAR(100)	Hub lokasi operasional (contoh: 'CIPUTAT')
created_at	TIMESTAMP	Waktu alokasi task

4. Tabel **paket** (Master Kiriman & Resi)

Kolom	Tipe Data	Keterangan
nomor_resi	VARCHAR(16)	Primary Key (8 karakter Base32 server-generated)
nama_barang	VARCHAR(150)	Deskripsi komoditas paket
pengirim	VARCHAR(100)	Nama pihak pengirim
alamat_pengirim	VARCHAR(255)	Alamat asal pengiriman
telepon_pengirim	VARCHAR(20)	Nomor kontak pengirim
penerima	VARCHAR(100)	Nama pihak penerima

Kolom	Tipe Data	Keterangan
alamat_tujuan	VARCHAR(255)	Alamat tujuan pengantaran
telepon_penerima	VARCHAR(20)	Nomor kontak penerima
berat_kg	DECIMAL(6,2)	Bobot paket dalam kilogram
jenis_layanan	ENUM	Opsi layanan: 'REGULER', 'EXPRESS', 'SAME_DAY'
status	ENUM	Status resi: 'DRAFT', 'TERDAFTAR'
created_by	VARCHAR(32)	Foreign Key pembuat paket <code>users(id)</code>
created_at	TIMESTAMP	Waktu resi dibuat

5. Tabel `scan_events` (Rekam Jejak Pemindaian Barcode)

Kolom	Tipe Data	Keterangan
scan_id	VARCHAR(32)	Primary Key log scan
nomor_resi	VARCHAR(64)	Foreign Key ke tabel <code>paket(nomor_resi)</code>
user_id	VARCHAR(32)	Foreign Key petugas pemindai <code>users(id)</code>
task_id	VARCHAR(32)	Foreign Key sesi tugas <code>tasks(task_id)</code>
waktu_scan	DATETIME	Timestamp presisi saat scanner membaca
lokasi	VARCHAR(100)	Hub / titik lokasi pemindaian
status_scan	ENUM	Hasil validasi: 'SUCCESS', 'DUPLICATE'
device_id	VARCHAR(50)	Identitas hardware / browser pemindai

Kolom	Tipe Data	Keterangan
-------	-----------	------------

`jenis_scan` ENUM Arah sortir: 'INBOUND', 'OUTBOUND'

`created_at` TIMESTAMP Waktu pencatatan database

6. Tabel `audit_logs` (Jejak Keamanan Sistem)

Kolom	Tipe Data	Keterangan
-------	-----------	------------

`log_id` BIGINT AUTO_INCREMENT Primary Key

`user_id` VARCHAR(32) Foreign Key pelaku aktivitas `users(id)`

`action` VARCHAR(100) Kode aktivitas (contoh: `SCAN_SUCCESS`, `TASK_COMPLETED`)

`details` TEXT Rincian keterangan aksi yang terjadi

`ip_address` VARCHAR(45) Alamat IP pemanggil (IPv4 / IPv6)

`created_at` TIMESTAMP Waktu audit tercatat

5. Alur Komprehensif Pemanggilan API: Front-End ke Back-End

Berikut adalah bedah rantai alur langkah demi langkah saat petugas menyelesaikan tugas shift kerja (Complete Task):

- [1. `PetugasScanPage.vue`] —> Klik tombol "Selesaikan Shift"
- [2. `taskStore.js`] —> Pinia trigger `completeTask(taskId)`
- [3. `task.service.js`] —> Format URL: `PATCH /api/tasks/{id}/complete`

[4. api.js] —> Axios suntikkan Header Authorization: Bearer <token>

[5. Nginx Proxy :8080] —> Deteksi rute "/api/", forward ke http://backend:5000

[6. uWSGI Server :5000] —> Alokasikan worker proses, parsing HTTP ke env PSGI

[7. web.pl & Router.pm] —> Regex matching rute & validasi Bearer Token

[8. TasksController.pm] —> Jalankan kueri "tasks_mark_completed" di query.sql

[9. MariaDB Database] —> Update status = 'SELESAI', set completed_at = NOW()

[10. Response Back] —> Kirim balik JSON { success: true } ke browser

Rincian Kode Implementasi Tiap Langkah:

Langkah 1: Komponen Antarmuka (**PetugasScanPage.vue**)

Komponen hanya menangani logika presentasi dan interaksi pengguna:

```
// src/pages/petugas/PetugasScanPage.vue
const executeFinishTask = async () => {
  if (!activeTask.value) return;
  const taskId = activeTask.value.task_id;

  // Memanggil aksi pada Pinia Store
  const result = await taskStore.completeTask(taskId);

  if (result.success) {
    $q.notify({
      type: 'positive',
      icon: 'task_alt',
      message: `Task ${taskId} berhasil diselesaikan!`,
      position: 'top'
    });
    router.push('/petugas/hasil');
  }
};
```

Langkah 2: State Management ([src/stores/taskStore.js](#))

Store mengisolasi status reaktif global dan memicu pembaruan state:

```
// src/stores/taskStore.js
export async function completeTask(taskId) {
  try {
    const result = await svcCompleteTask(taskId);
    if (result.success) {
      // Re-fetch data tugas terbaru agar tampilan dashboard sinkron
      await this.fetchTasks();
    }
    return result;
  } catch (err) {
    return { success: false, message: err.message };
  }
}
```

Langkah 3: Lapisan Service I/O ([src/services/task.service.js](#))

Service menentukan kontrak endpoint dan metode transmisi HTTP:

```
// src/services/task.service.js
export async function completeTask(taskId) {
  // Menggunakan HTTP PATCH karena hanya memperbarui kolom status tugas
  const data = await api.patch(`/api/tasks/${taskId}/complete`);
  return { success: true, message: data.message };
}
```

Langkah 4: Klien HTTP Terpusat ([src/services/api.js](#))

Axios bertindak sebagai pengatur transmisi jaringan dan penyedia token keamanan otomatis:

```
// src/services/api.js
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || "",
  timeout: 15000,
  headers: { 'Content-Type': 'application/json' }
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('wahana_token');
```

```

if (token) {
    config.headers.Authorization = `Bearer ${token}`;
}
return config;
});

```

Langkah 5: Penerusan oleh Reverse Proxy Nginx (docker-nginx.conf)

Nginx menerima paket HTTP pada port `8080` dan meneruskannya ke kontainer backend:

```

location /api/ {
    proxy_pass http://backend:5000/api;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

```

Langkah 6 & 7: Pemrosesan di uWSGI, Entrypoint, dan Router Perl

uWSGI memanggil `backend/web.pl` yang mengurai hash `$env`, lalu mengoper ke `Wahana::Router`:

```

# backend/lib/Wahana/Router.pm
my @ROUTES = (
    ...
    [ 'PATCH', qr{^/api/tasks/([^\s]+)/complete$}, \&Wahana::Controller::TasksController::complete, { auth => 1 } ],
    ...
);

```

- Router mencocokkan URL melalui Regex dan mengekstrak parameter `$captures->[0]` (yaitu ID Task).
- Metadata `{ auth => 1 }` memicu `Wahana::Auth->authenticate_request()` untuk memverifikasi keabsahan tanda tangan token.

Langkah 8 & 9: Eksekusi Bisnis & Pembaruan Database ([TasksController.pm](#))

Controller mengeksekusi instruksi data secara aman:

```

# backend/lib/Wahana/Controller/TasksController.pm
sub complete {
    my ($req, $captures) = @_;
    my $task_id = $captures->[0];
}

```

```

my $user_id = $req->{auth_user}{uid};

my $dbh = Wahana::Db->connect();

# Eksekusi kueri terpusat dari query.sql
$dbh->do(
    Wahana::Query->get('tasks_mark_completed'),
    undef, $task_id
);

# Catat audit log aktivitas
record_audit(
    user_id => $user_id,
    action => 'TASK_COMPLETED',
    details => "Task $task_id berhasil diselesaikan."
);

return { success => \1, message => "Task $task_id telah diselesaikan." };
}

```

Langkah 10: Siklus Pengembalian Respon (Response Back)

1. Controller mengembalikan objek hash Perl `{ success => 1, ... }`.
2. Router membungkusnya ke dalam format teks JSON dan menyematkan header HTTP `200 OK` melalui `Wahana::Response::json_response()`.
3. uWSGI mengirimkan payload JSON melewati jaringan virtual ke Nginx.
4. Nginx meneruskannya ke browser.
5. Axios Interceptor menerima data, Pinia memperbarui memori state, dan notifikasi hijau sukses muncul di layar petugas.

6. Tata Kelola Konfigurasi & Panduan Operasional DevOps

A. Variabel Lingkungan (`.env`)

Semua parameter konfigurasi sensitif dipisahkan dari kode program: * `DB_HOST`: Nama host service database Docker (`db`). * `DB_PORT`: Port database internal (`3306`). * `DB_NAME`: Nama skema basis data

([wahana_scan](#)). * [DB_USER](#) / [DB_PASS](#): Kredensial akun database aplikasi ([wahana_app](#) / [wahana_pass](#)). * [WAHANA_API_PORT](#): Port uWSGI HTTP socket (5000). * [VITE_API_BASE_URL](#): URL basis API front-end (dikosongkan agar same-origin proxy Nginx aktif). * [VITE_USE_LOCAL_DATA](#): Saklar mode [false](#) untuk live backend Perl atau [true](#) untuk mode demo offline.

B. Perintah Pengoperasian Sistem

Proyek dilengkapi script otomatisasi shell untuk kemudahan tim operasional: * **Menyalakan Seluruh**

Sistem: [bash ./docker-up.sh](#) # Atau secara manual: [docker compose up -d --build](#) * **Mematikan Sistem:** [bash](#)

[./docker-down.sh](#) # Atau secara manual: [docker compose down](#) * **Melihat Log Transaksi Backend secara Real-**

Time: [bash docker logs -f wahana_scan_backend](#) * **Membuka Dokumentasi API Interaktif:** Kunjungi peramban di alamat: <http://localhost:8080/api/docs/>

7. Kesimpulan Arsitektur

Platform **Dijak Express** mendemonstrasikan integrasi sistem yang matang: 1. **Keandalan Tinggi:** Front-End PWA modern berbasis Vue 3 menjamin pengalaman pengguna yang instan dan bebas lag saat memindai barcode. 2. **Kinerja & Efisiensi Memori:** Backend Perl yang ditenagai oleh uWSGI Application Server menghasilkan throughput pemrosesan transaksi tinggi dengan konsumsi RAM yang sangat minim. 3. **Keamanan Bertingkat:** Mulai dari isolasi jaringan Docker bridge, otentikasi Bearer Token RBAC, hingga eliminasi mutlak celah SQL Injection berkat arsitektur *External Query Catalog* pada [Wahana::Query](#).